# SFWRENG / MECHTRON 3K04 Software Development

## Today: Software Development Styles

McMaster University | Engineering
Computing & Software

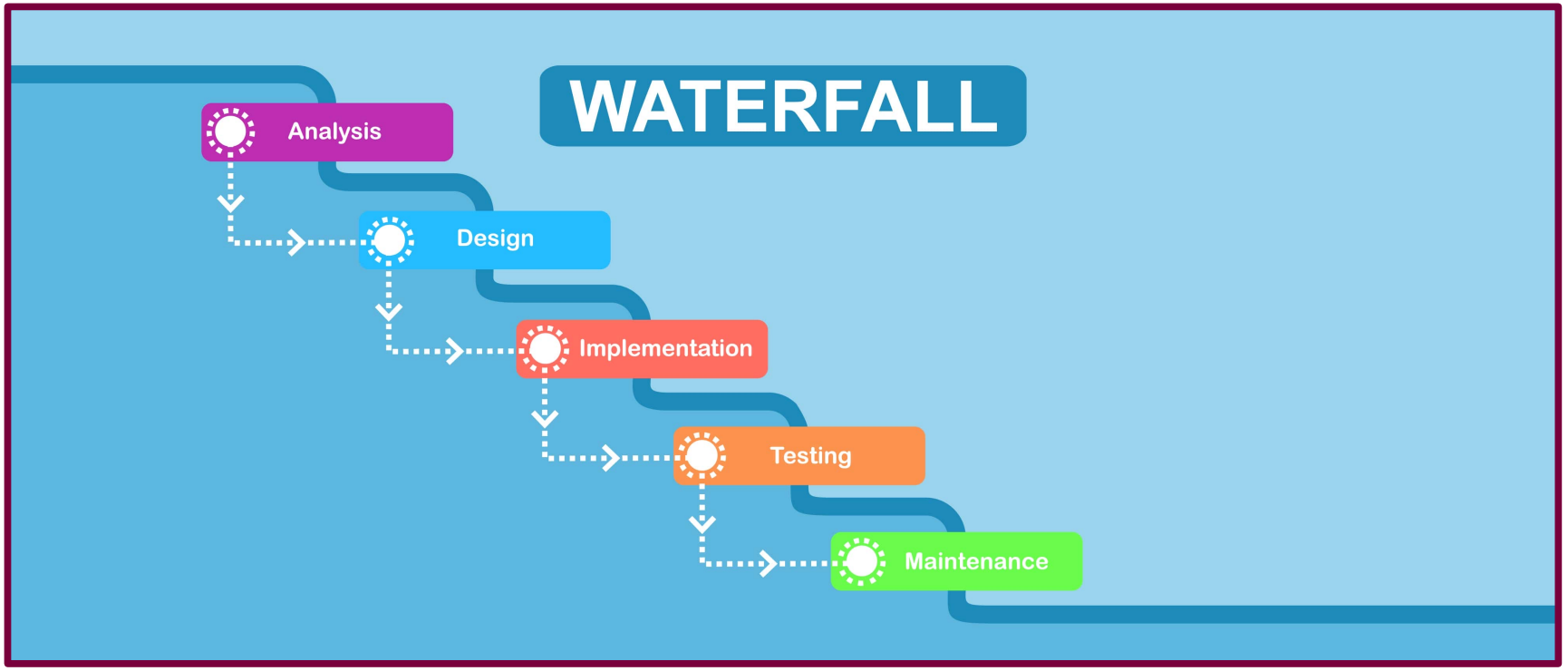
September 15th, 2026

Dr. Angela Zavaleta Bernuy

# Software Development Styles

- How do you **approach** software development?
- How do you **scope** software?
- How do you **support** software?
- How do you know when **deliver** software?
- Today: Let's look at the many Software Development Life Cycle (SDLC) methodologies / models!

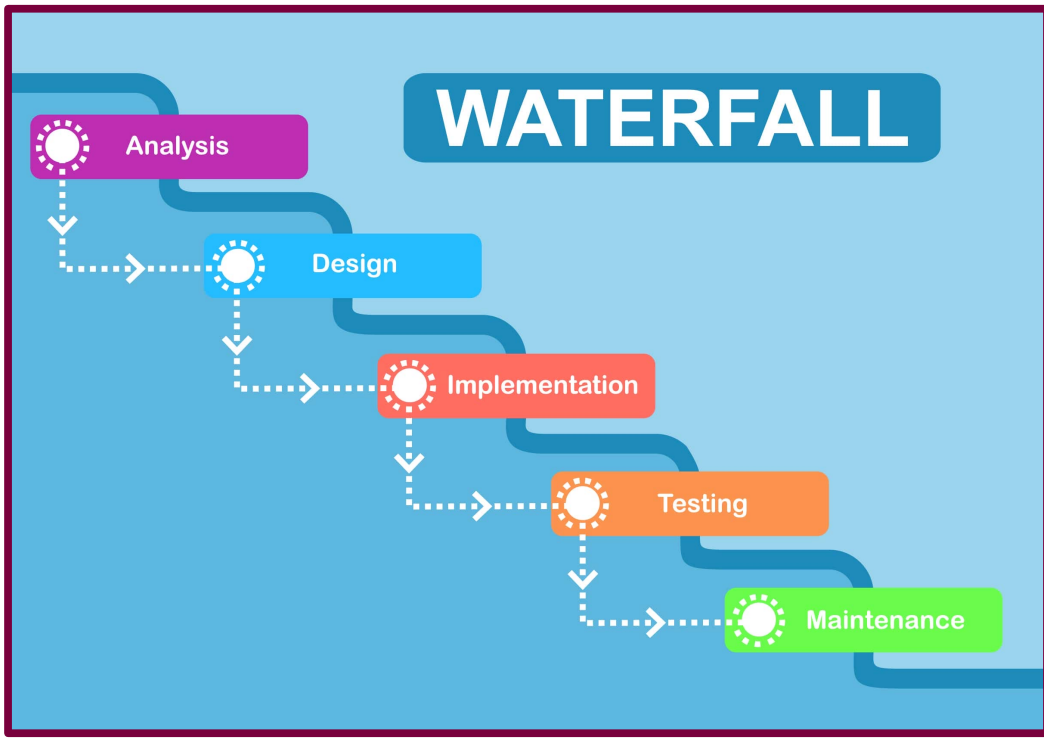
# (Traditional) SDLC: Waterfall Model




https://www.chetu.com/blogs/technical-perspectives/understanding-software-development-life-cycle.php

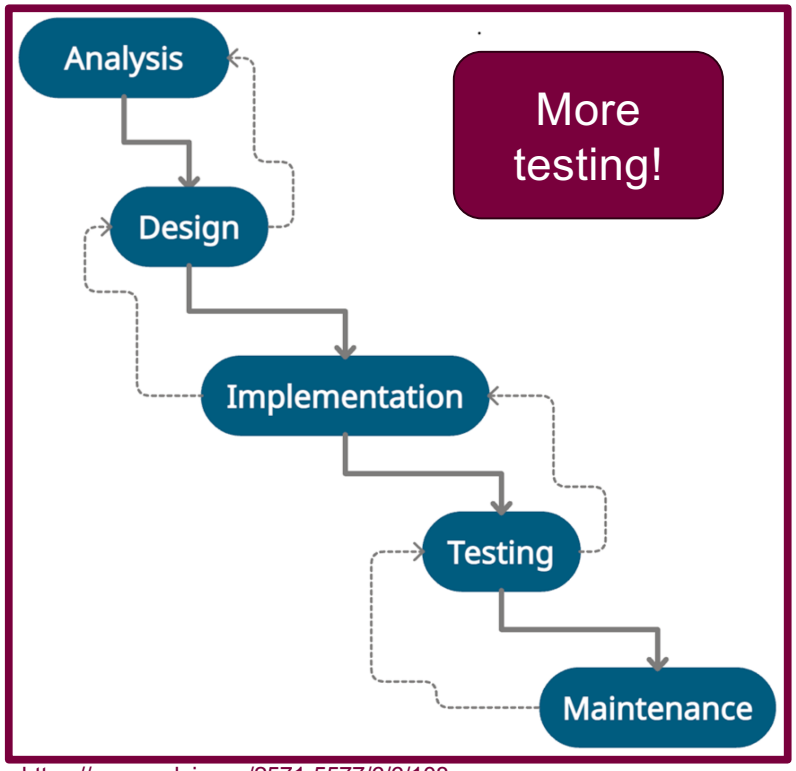
# (Traditional) SDLC: Waterfall

- How do you **approach** software development?
  - Development is supported in sequential phases
  - Each phase must be completed before moving to the next
- How do you **scope** software?
  - In the requirements phase
  - A formal requirements baseline is set
- How do you **support** software?
  - In the maintenance phase
- How do you know when **deliver** software?
  - All requirements have been met and verified
  - Delivered in a single, final release




https://www.chetu.com/blogs/technical-perspectives/understanding-software-development-life-cycle.php

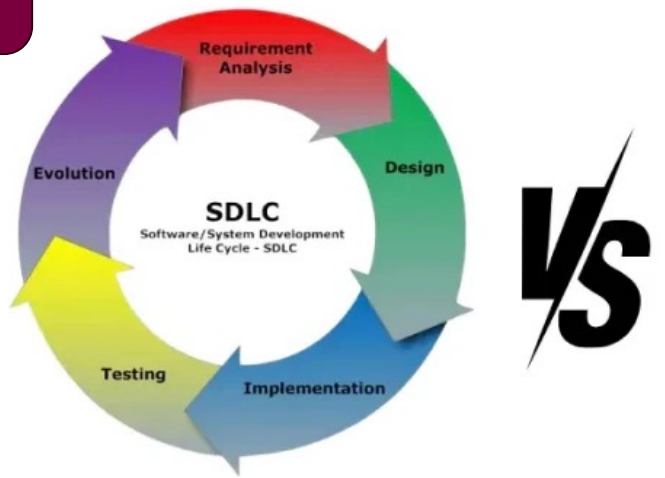
# (Traditional) SDLC: Modified Waterfall




https://www.mdpi.com/2571-5577/6/6/108

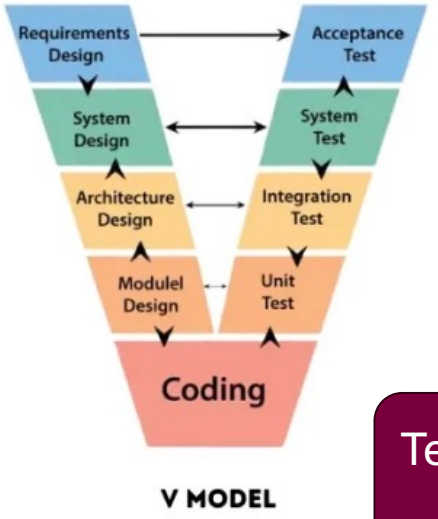
# (Traditional) SDLC: V Model

Testing AFTER development







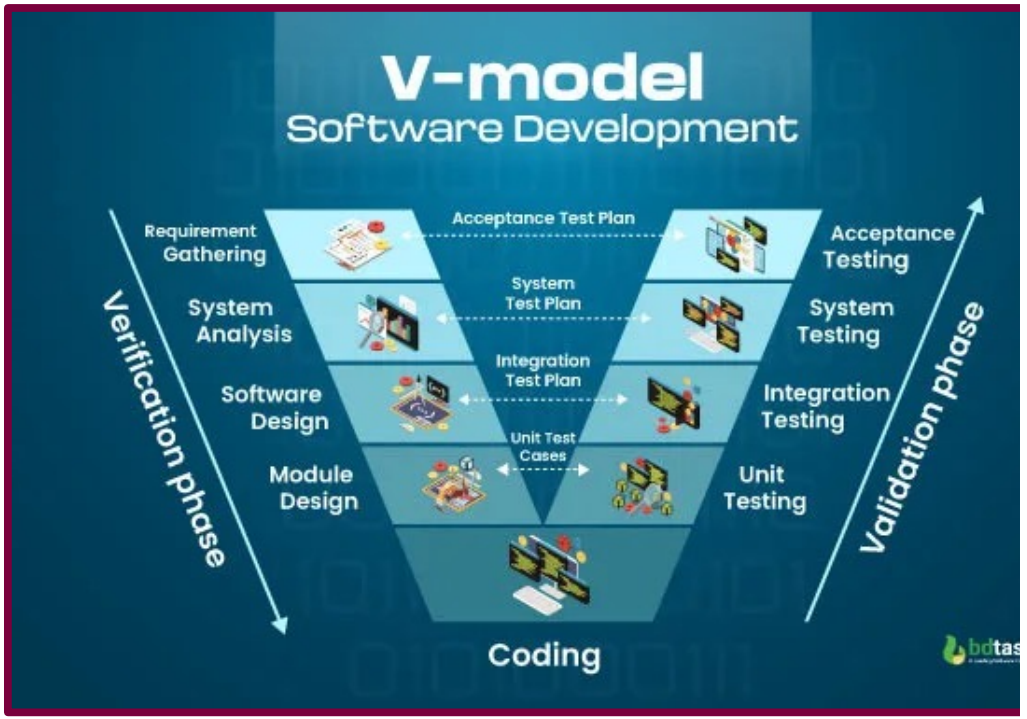



Testing ALONGSIDE development

https://stepmediasoftware.com/blog/v-model-and-waterfall-model/

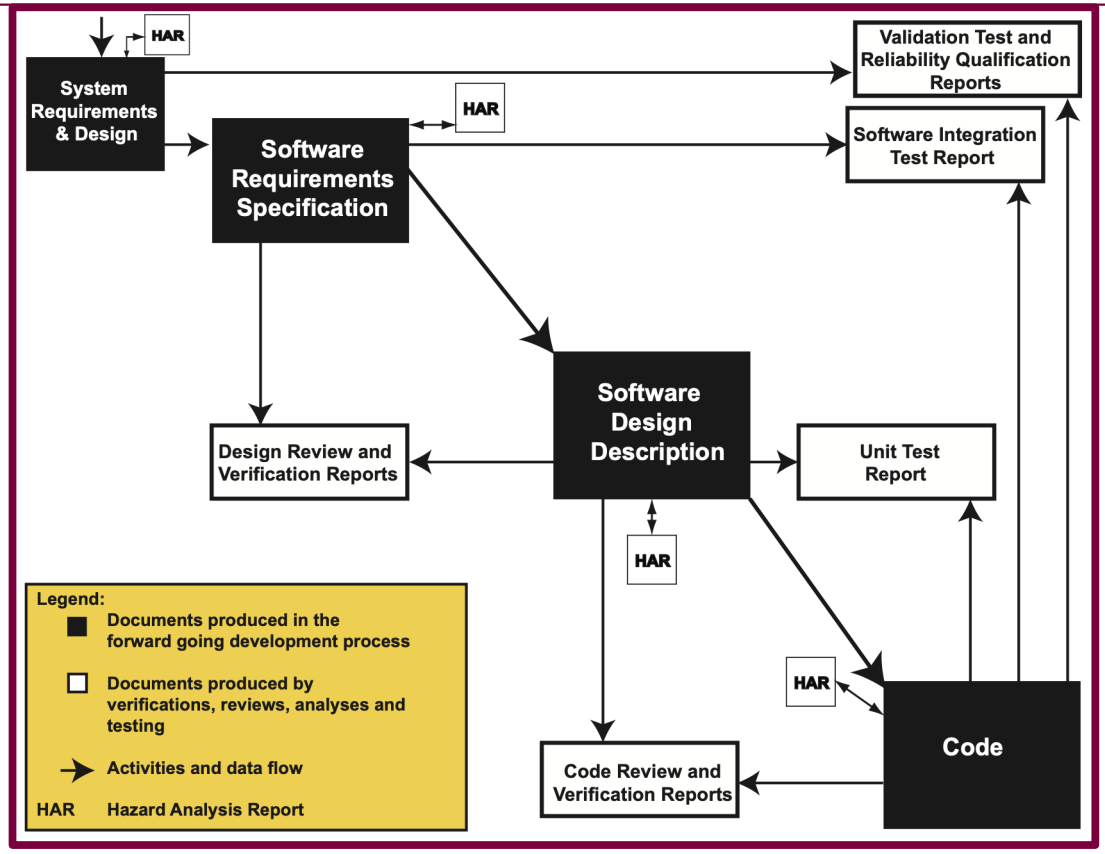
# (Traditional) SDLC: V Model

- How do you **approach** software development?
  - Development is supported in sequential phases
  - Testing activities are planned in parallel
- How do you **scope** software?
  - In the requirements phase
  - A formal requirements baseline is set
- How do you **support** software?
  - In the maintenance phase
- How do you know when **deliver** software?
  - All requirements have been met and verified
  - Delivered in a single, final release



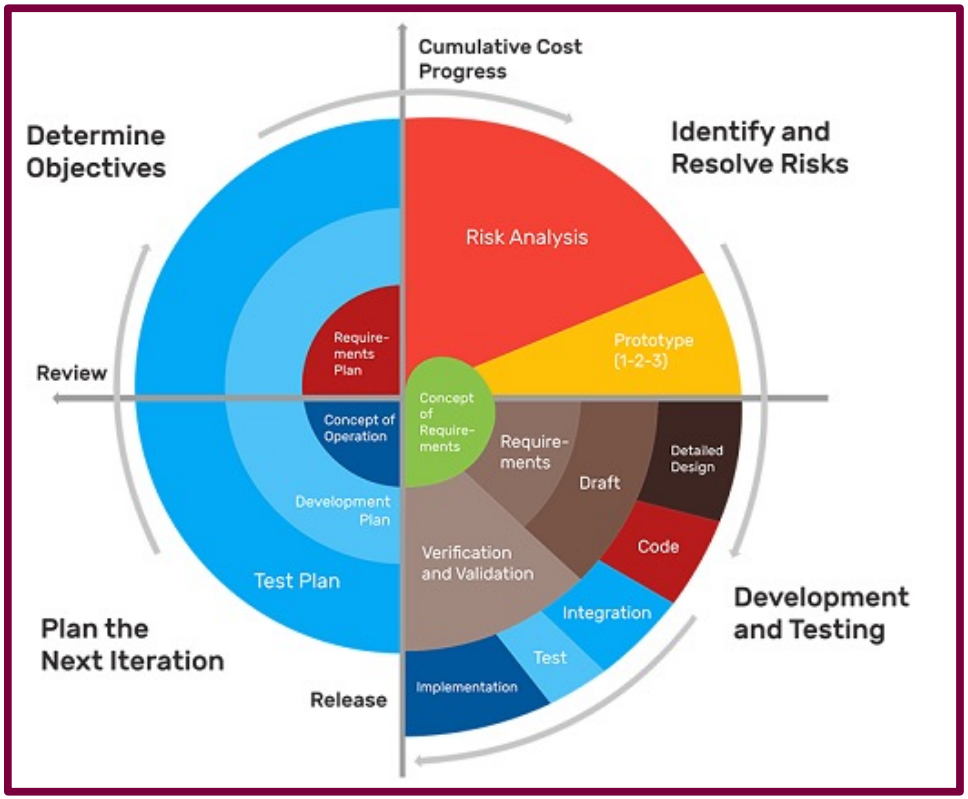

https://www.bdtask.com/blog/v-model-in-software-development

# (Traditional) SDLC: Assurance in Critical Env.




https://www.cl.cam.ac.uk/archive/mjcg/FMStandardsWorkshop/talks/MLawford.pdf

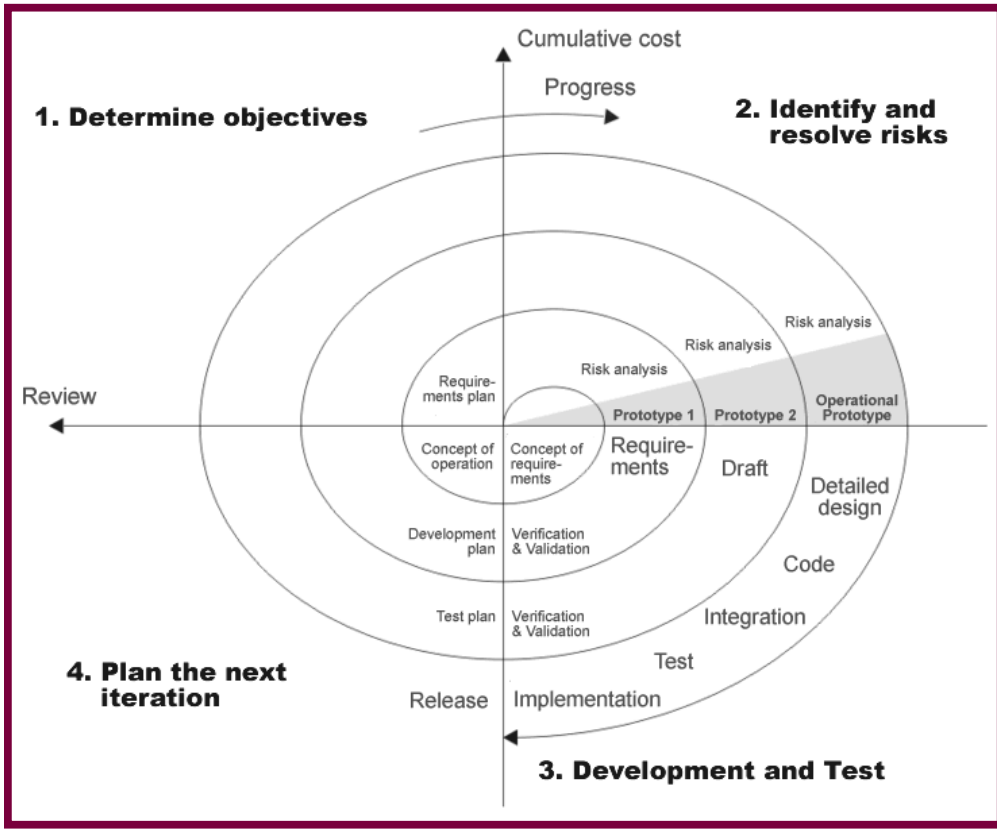
# (Traditional) SDLC: Spiral Model



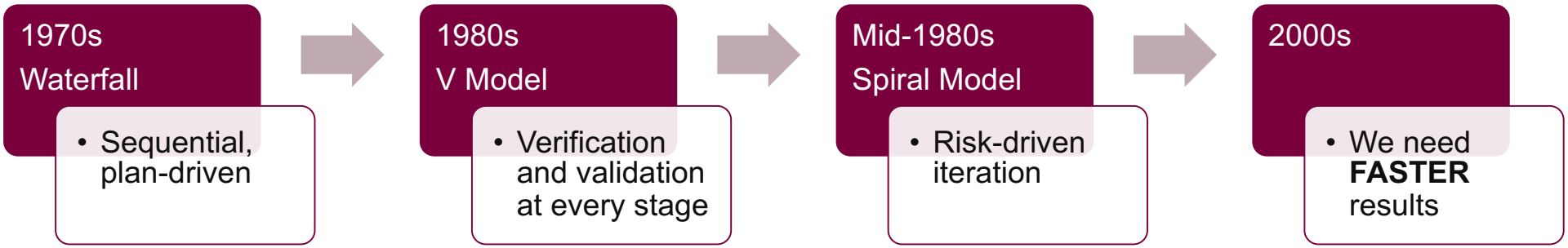

https://www.sovereignconsult.com/blog/spiral-model-methodology/.

# (Traditional) SDLC: Spiral Model

- How do you **approach** software development?
  - Development is supported in iterative cycles
  - Planning, risk analysis, engineering, and evaluation
- How do you **scope** software?
  - Scope is defined incrementally at every cycle
  - Adapt to changing needs!
- How do you **support** software?
  - In the maintenance phase, but also during development
- How do you know when **deliver** software?
  - It can be incremental!

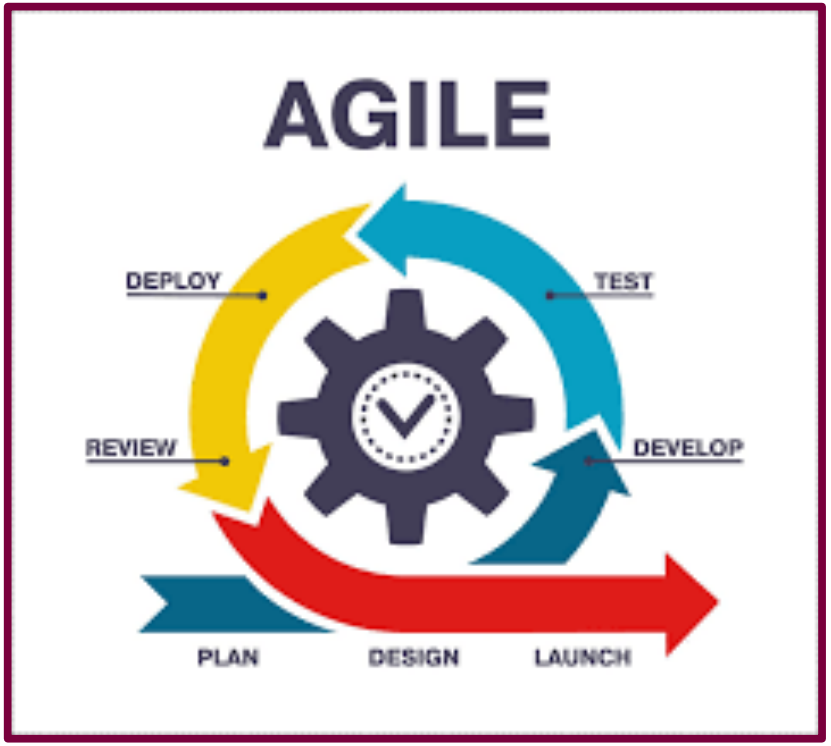


# Let's get into the timeline...



# SDLC: Agile

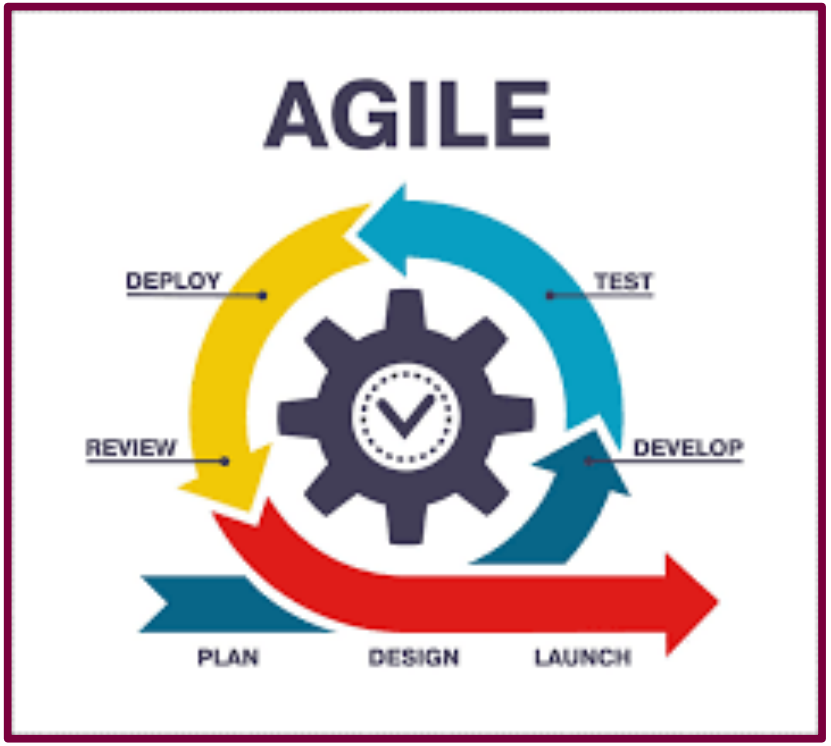
- Iterative and flexible approach that breaks projects into small, manageable cycles (*iterations*)
- More suitable to address changes in software development
- Incremental development, successive release of products
- Follow the Agile Manifesto principles: https://agilemanifesto.org/
- Different versions:
  - SCRUM
  - Extreme Programming (XP)
  - Kanban
  - Etc.




https://yuktisolutions.com/blog/agile-sdlc-for-your-project

# SDLC: Agile

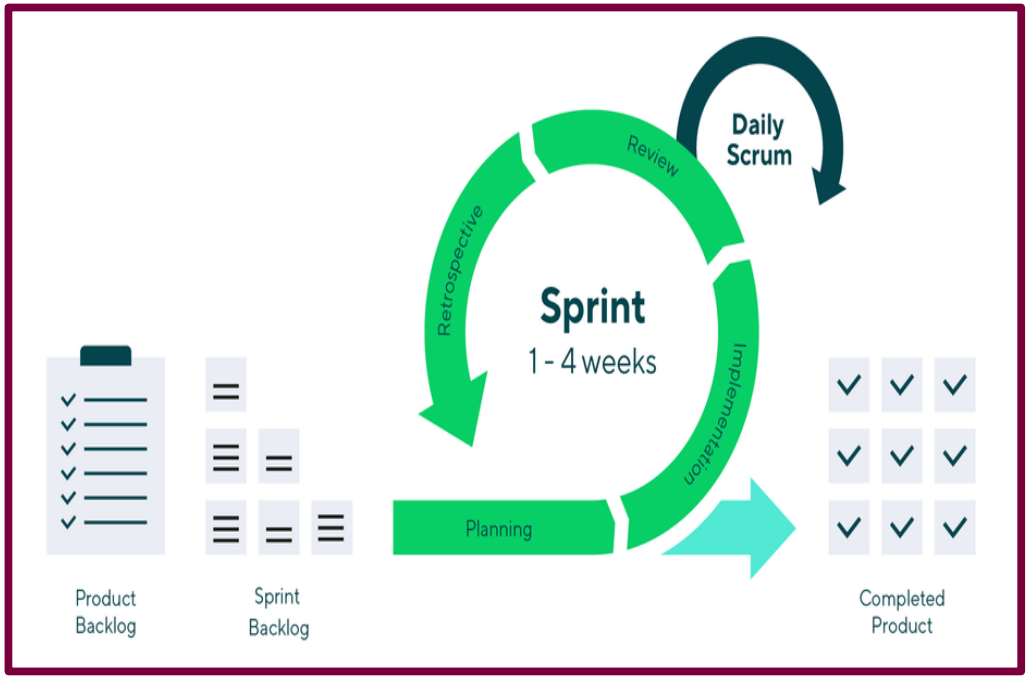
- How do you **approach** software development?
  - Iterative and incremental
  - Work is divided into short cycles (iterations)
- How do you **scope** software?
  - Managed in a prioritized backlog of features or user stories
  - Requirements are not all fixed upfront
- How do you **support** software?
  - Continuous support and ongoing improvement
- How do you know when **deliver** software?
  - Early and often
  - Working, tested and valuable




https://yuktisolutions.com/blog/agile-sdlc-for-your-project

# SDLC: Agile – SCRUM

- Origin of the word: rugby
  - Players huddle and work together to move the ball forward
  - Which is what Agile teams should do!
- Users involved:
  - Product owner
  - Scrum master
  - Other members
- Workflow:

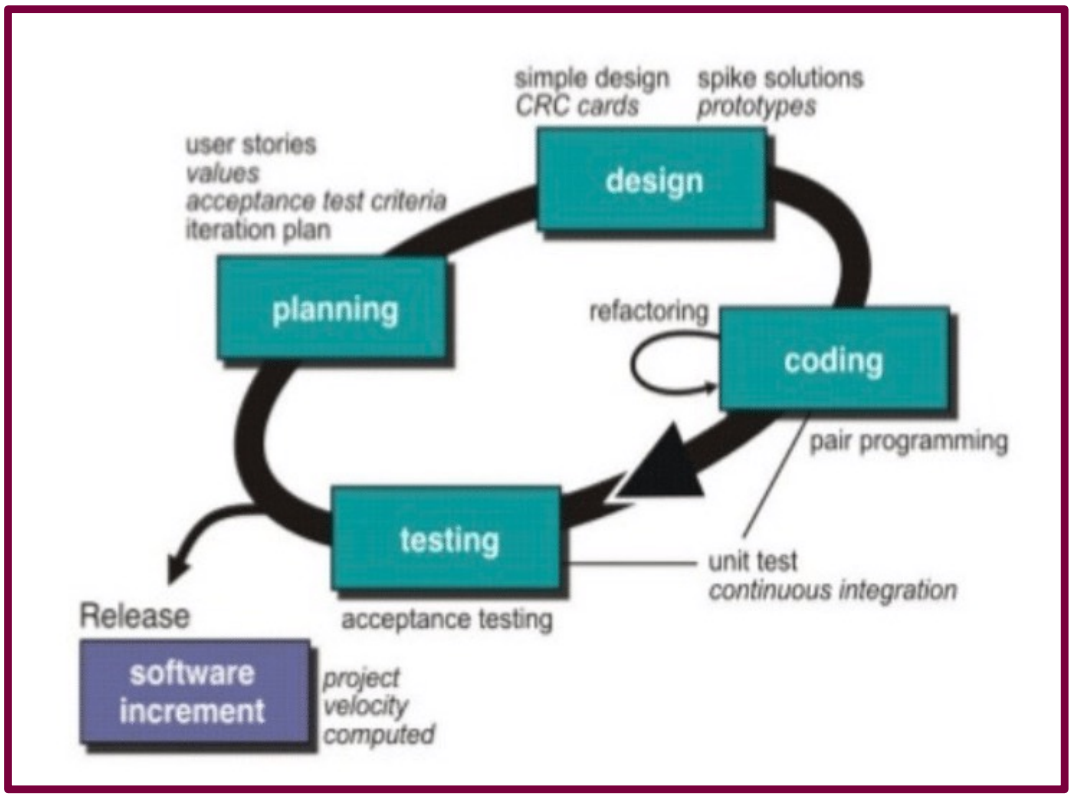



https://www.wrike.com/agile-guide/agile-development-life-cycle/

# SDLC: Agile – Extreme Programming (XP)

- XP considers four basic activities: coding, testing, listening and designing
- Coding: "*Without code, there is no working product*"
  - Practices: pair programing, code review
- Testing: Test-driven development, continuous testing
- Listening:
  - Planning game: short planning session each iteration, typically once a week
  - Release planning: set goals for near-term releases
  - Iteration planning: details tasks for developers during the next short cycle
- Designing
  - Simple design: build the simplest system that works, then improve it continuously
  - Refactoring

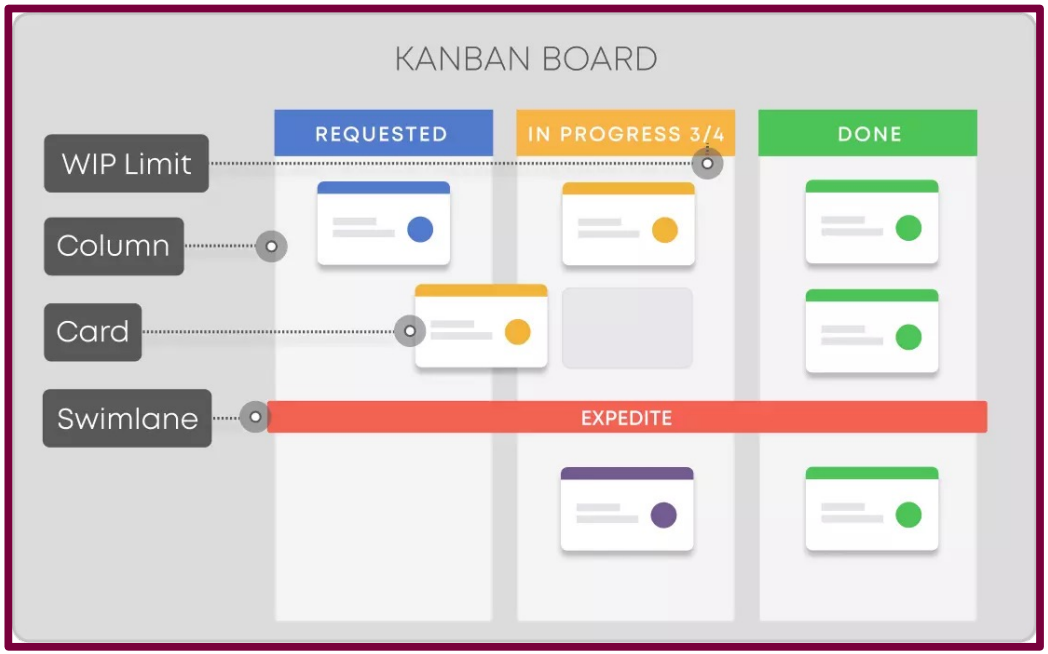
# SDLC: Agile – Extreme Programming (XP)




https://www.researchgate.net/figure/Extreme-Programming-XP_fig2_356185795

# SDLC: Agile – Kanban

- ”Visual card” in Japanese
- Focuses on visualize work, limit work in progress and improve flow
  - Visualizing work: Kanban board
  - Only allow a small number of tasks per column
  - Track how smoothly tasks move across
- Make process policies explicit
- Continuous improvement



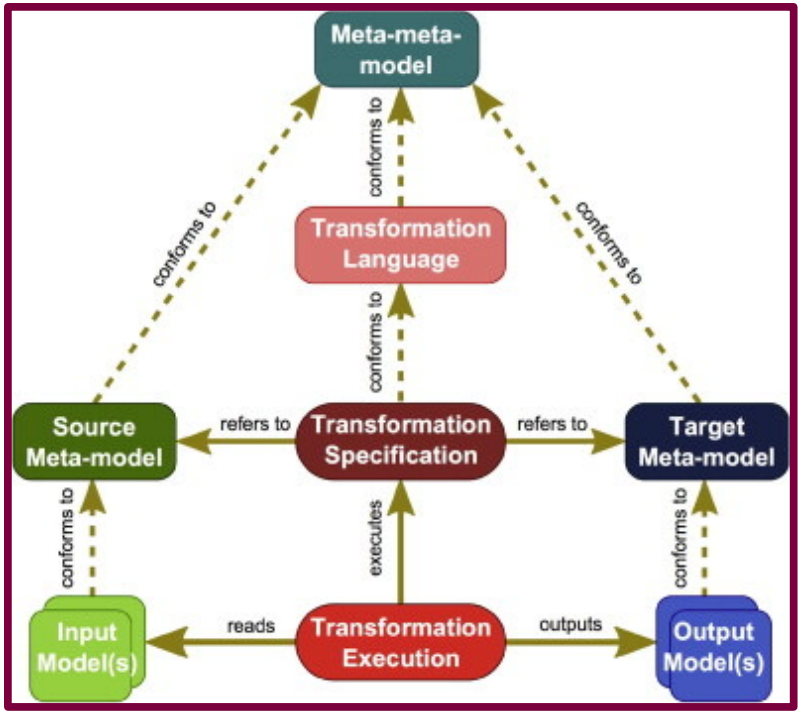

https://businessmap.io/kanban-resources/getting-started/what-is-kanban-board

# Modern Trends in SDLC

- Agile:
  - Meant for smaller teams and smaller development cycles
  - Some people are distrustful of how Agile can be used to handle safety critical engineering
- Waterfall
  - Meant for bigger teams and bigger projects
  - Some people dislike how long it takes to develop using waterfall
  - Tried and tested for development of critical components
- We need a methodology that can fit in the SDLC to make development of safety critical systems efficient
  - Model-Driven Engineering!

# Model-Driven Engineering (MDE)

- It makes models the main artifact of development instead of raw code
  - Code can be generated automatically from these models
  - E.g., we can transform a UML diagram into Java classes, or a state machine model into embedded system code ;)
- Goal: raise the level of abstraction so developers think in **domain concepts** rather than low-level programming
- Reduce manual coding and relies on code generation and automation tools
- **Is MDE an SDLC?**




https://www.sciencedirect.com/topics/computer-science/model-driven-engineering

# MDE in Safety-critical Systems

- **Assurance**: Making statements about properties of a system and demonstrate that we have some adequate level of confidence that statements are true.
- In safety-critical systems, we want to *assure* that the system is safe, secure and dependable
- Traditionally, assurance was implicit
  - Developers would provide assurance by providing the documentation and test results of the system
- With MDE, assurance can be explicit
  - We will talk about this throughout the course

# Takeaways!

- A number of SDLC models are being used and have been transformed across the years
  - Our focus is safety critical systems, for which spiral and V model are used often
  - In particular, the spiral model emphasizes the iterative nature of hazard analysis
  - Of course, we can use other SDLC models, but keep hazard/risk analysis in mind
- MDE is prevalent in many industries, specially for safety-critical systems
- Explicit assurance is needed to ensure the systems we develop are safe, secure and dependable

# Questions?

# SFWRENG / MECHTRON 3K04 Software Development

## Today: Working in Teams

McMaster University | Engineering
Computing & Software

September 17th, 2026

Dr. Angela Zavaleta Bernuy

# Where We Are

## SDLC models

Waterfall, V-Model, Spiral, Agile, MDE — the process.

## Working in Teams

Every one of those processes is executed by a group of people.

## Your project

The application of both!

# Why Teams?

# From Real-life to Software

- **Example:** A sports team
  - Goalkeeper, defenders, strikers.
  - Nobody plays every position, but everyone plays toward the same goal.
- **Example:** A professional kitchen
  - Head chef, line cooks, pastry, front of house.
  - A dish is only “done” when the whole line agrees it is.
- Software is no different!
  - Requirements, design, firmware, UI, testing, integration.
  - Different roles, one product, one definition of done.

# Let's Chat: Concerns Around Teamwork?

- **Week 1 poll:**
  - Worried about having a good and reliable team
- **Discuss:**
  - What is the worst team experience you've had on a course or co-op project?
  - What specifically went wrong: people, process, or communication?
  - What would have prevented it?
- These are things to consider when writing the team contract!

# What Research Says: Berrezueta-Guzman (2025)

**23**

student teams, semester-long project, six incremental assignments

**−0.83**

correlation between contribution discrepancy and project grade

**~90%**

mean project grade for teams with ≤10% contribution discrepancy

**~60%**

mean project grade for teams with >20% contribution discrepancy



Berrezueta-Guzman, S., Parmacli, I., Habib, M. K., Krusche, S., & Wagner, S. (2025, April). Assessing teamwork dynamics in software development projects. In *2025 IEEE Global Engineering Education Conference (EDUCON)* (pp. 1-5). IEEE.

# What Research Says: Berrezueta-Guzman (2025)

- **Struggling teams:** (>20% discrepancy)
  - Unclear role definitions
  - Uneven task distribution
  - Weak or irregular communication
  - Misaligned self-perception (members overestimated their own contribution)
  - Conflict left unaddressed
- **Strong teams:** (<10% discrepancy)
  - A leader chosen for organizational skills and expertise
  - 1-2 regular meetings per week (plus a chat channel)
    - Lab time ☺
  - Open-discussion approach to conflict
  - Deliberately balanced workload
  - Realistic self-assessment and regular reflection

# Tuckman's Stages of Team Development

## 1. Forming

Polite, uncertain, avoiding conflict. Nobody knows who's good at what.

## 2. Storming

Disagreements about approach, roles and effort surface. Friction is highest.

## 3. Norming

Agreed ways of working, Roles settle. Trust starts to form.

## 4. Performing

The team works with minimal friction and self-corrects.

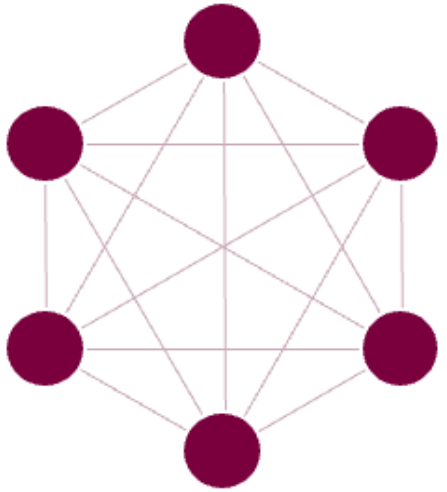


Tuckman, B. W. (1965). Developmental sequence in small groups. *Psychological bulletin*, *63*(6), 384.

# Brook's Law

*"Adding manpower to a late software project makes it later."*

- Software work is not perfectly partitionable
  - New people need onboarding, and everyone needs to talk
- Communication paths grow as n(n−1)/2, not n
  - 5 people → 10 paths.  6 people → 15 paths.  10 people → 45 paths.
- Why your project team is 4-5 people and not 10
- Why sub-teams of 2–3 with a clear interface between them work better than "everyone on everything"
- Why you cannot fix a late deliverable by "just all working on it" the night before





Brooks, F. P., & mythical Man-Month, T. (1975). Essays in Software Engineering. *Adisson-Wesley, Reading, Mass*.

# Conway's Law

*"Organizations design systems mirror their own communication structure."*

- If two people never talk, the two modules they own will have a poorly designed interface
- If one person owns everything, you get a bus/ladder factor of 1
- Flip it around: decide the module boundaries first, then assign owners
  - And make the owners talk



Conway, M. E. (1968). How do committees invent. *Datamation*, *14*(4), 28-31.

# Clear Roles

# Roles in a Software Team

- Backend / firmware
  - Data, logic, hardware interface
- Frontend / UI
  - What the users sees and interacts with
- QA / Testing
  - Owns the definition of done
- Design / UX
  - Workflows, safety of interaction
- Product / Project management
  - Scope, priorities, schedule

**Scrum Roles**

- Product owner
  - Owns the backlog and priorities. Says what “valuable” means.
  - For you: the person who keeps re-reading the project spec.
- Scrum master
  - Owns the process: runs meetings, removes blockers, protects the team's focus. Not the boss.
- Development team
  - Cross-functional; collectively responsible for delivering working, tested increments.

# Suggested Roles

- Coordinator
  - Runs meetings, tracks the kanban board, etc.
- Requirements and documentation lead
  - Owns the requirements doc and traceability. Keeps the spec honest.
- UI lead
  - Owns the application user-facing side.
- Logic lead
  - Owns the application logic.
- Testing lead
  - Owns the definition of done, test plan, etc.
- Integration lead
- Repository lead

# Suggested Roles: RACI Matrix

- **Responsible (R):** The person who does the actual work to complete the task.
- **Accountable (A):** The single person who owns the final result and signs off on the work.
- **Consulted (C):** Subject matter experts who provide input and advice before a decision or step is finished.
- **Informed (I):** People who are kept updated on progress after decisions or milestones happen

| Deliverable 1 | Coord. | Req. Lead | Test Lead | Etc. | | |
|---|---|---|---|---|---|---|
| Requirements document | I | R | A | | | |
| UML | I | C | A | | | |
| Specifications | I | A | C | | | |

# Meetings That Actually Work

## Weekly sync (30–45 min)

Fixed slot, same place, everyone. -> Labs!

Agenda discussed the day before.

Ends with: who does what by when, written down.

One person takes notes; notes go in the repo.

## Stand-up (5–10 min, 2×/week)

Each person, in turn:

1. What I finished
2. What I'm doing next
3. What's blocking me

No problem-solving in the stand-up, park it and take it offline with the people who need to be there.

## Working session

Actually building together: pairing, integration, debugging.

Schedule it. "*We'll meet when we need to*" means you won't.

Lab time counts, use it!

# *If it isn't written down, it didn't happen.*

- **Tasks**
  - Issue tracker / Kanban board (GitHub Projects). Every task has one owner and a due date.
- **Decisions**
  - A DECISIONS.md (or wiki page): what was decided, why, and the alternatives rejected. Regulators will ask for this later; TAs will ask for it now.
- **Knowledge**
  - README for setup, wiki for how-tos. If only one person knows how to do X, document it immediately.
- **Conversation**
  - Chat is ephemeral. Anything that changes a task, a decision or a deadline gets copied to the place above.

# Phycological Safety

- Definition: team members feel safe to take interpersonal risks
  - Ask a “dumb” question, admit a mistake, disagree with the lead of a task
- Past work: It mattered more than who was on the team, how smart they were, or how senior
- Why it matters for safety-critical software specifically:
  - The person who noticed the bug has to feel able to say so
  - Example: Therac-25 and Boeing 737 MAX both involved people who knew and didn't speak up loudly enough.
- How you build it in a student team:
  - Respond to questions with answers, not eye-rolls
  - Say “I don't know” and “I was wrong” out loud, especially if you're the strongest coder
  - Criticize the task, never the person; give hard feedback privately

# Caravana's *Simple Guide to Teamwork*

**Be kind and respectful.**

Treat teammates the way you'd want to be treated on a bad day.

**Stay humble and keep learning.**

Experience varies; the new to software person on your team may know the thing you don't.

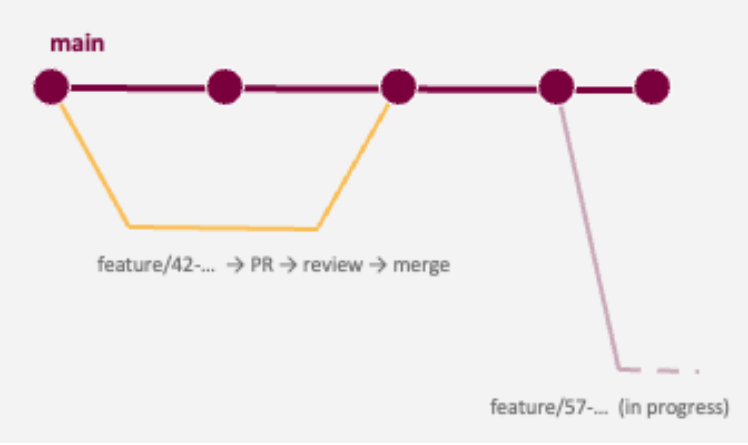
# Caravana's *Simple Guide to Teamwork*

1. Greet people. It's cheap and it compounds
2. Help when asked, if you have the capacity
3. Acknowledge good work out loud
4. Show up to every meeting
5. Camera on, mic on, *actually* present when remote
6. Explain patiently; rephrase, don't repeat louder
7. Do what you said you'd do, by when you said
8. Raise concerns privately, praise publicly
9. Value opinions that differ from yours
10. Seniority is earned by being reliable, not by being loud

# Tools & Workflow

# Git!

- main is always demo-able. Nobody commits directly to it.
- One branch per task, named after the issue: feature/42-access
- Small, frequent commits with messages that say why, not just what
- Pull requests are how work gets into main, and every PR is reviewed by someone who didn't write it
- Pull and rebase (or merge) from main daily so integration is a non-event, not a weekend



# Code Review

**Bad:** “this is wrong”

**Bad:** “why didn't you just use a switch?”

**Good:** “If pacing_mode is VVI and the sensor times out here, we fall through to line 84 and never set the flag. Should this branch return early?”

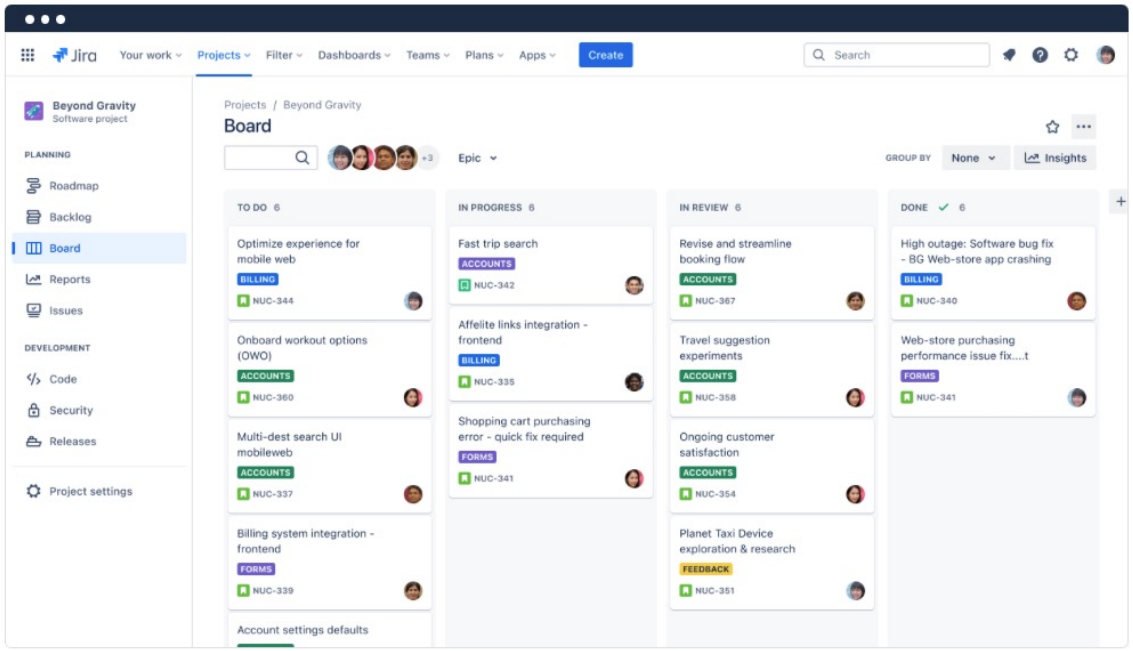
Review the code, not the coder. Ask questions. Point at lines. Say what you'd accept. Reply to every comment, and resolve it before merging.

# Pair Programming

- Anything one person on the team is the only one who understands
- The tricky state machine, the integration between two people's modules
- Onboarding a teammate into an area they haven't touched

Driver types, navigator thinks ahead and reviews. Swap every 20–30 minutes. Both names on the commit.

# Kanban




**Rules:** every card has one owner · cards move left-to-right only · WIP limit on In progress · a card is Done only when it meets the definition of done (next slide) · the board is reviewed at every stand-up.

# What does DONE mean?

- "*I finished the dispensing module*" means very different things to the person who wrote it and the person who has to integrate it at 11pm the night before the demo.
- A shared definition of done is the best way to prevent that conversation.
- It is also exactly what a regulator means by "evidence."
- Done should mean the same to all of you:
  - Demoed to at least one other teammate
  - Code merged to main via a reviewed PR
  - Unit tests written and passing
  - Requirement it satisfies is linked in the PR / issue
  - Documentation updated (README, design doc, or wiki)


When things go wrong

# The Four Failure Modes and Fixes

- Miscommunication
  - Ex: Two people build to different assumptions; the modules don't fit.
  - **Fix:** Written decisions, regular meetings, one source of truth for tasks.
- Unclear responsibilities
  - Ex: “I thought you were doing that.” Work falls through the cracks or gets done twice.
  - **Fix:** RACI per deliverable; one owner per card; exactly one accountable.
- Conflict
  - Disagreement turns personal; people stop talking; the team splits.
  - **Fix:** Focus on the problem, not the person. Decide with data. Escalate early.
- Hero dependency
  - One person carries the team (or hoards the work). Ladder/Bus factor = 1. Grades suffer for everyone.
  - **Fix:** Pair, rotate, document. Track contribution and rebalance early.

# Uneven Contribution: Spot it Early and Fix it!

- Weeks 3-4: nothing is visible yet, so nobody notices. Week 8: it's too late to rebalance.
- Look at the evidence, not the vibes: commit history, PRs reviewed, cards moved, meetings attended
  - Your TAs look at the same evidence.
- Misaligned self-perception is common: people honestly believe they did more than the log shows. Show the log, without blame.
- Usually it isn't laziness. It's being stuck, overwhelmed, or unsure what to do. Ask first.
- Rebalance in the open at a meeting; write the new split down
- If it persists: the escalation path you agreed in the contract (team → TA → me), early, with the evidence.

**A script for the hard conversation:**

*"Hey — I noticed the authorization card has been In progress for two weeks and there's nothing on the branch yet. Is something blocking you? Do you want to pair on it Thursday, or should we move it to someone else and give you the test plan instead?"*

# Handling Disagreements

- Focus on the problem
  - "*The UI button for save is not working and removes the data*" vs. "*your UI code is bad.*"
  - Separate the person from the position.
- Decide with data
  - Prototype various options for an hour. Measure. Check it against the requirement.
  - If you still disagree, the accountable person for that area (the A in RACI) decides, and everyone commits.
- Escalate early
  - Raise it in the team, at a meeting, not in chat at midnight.
  - Still stuck after a week? Bring it to your TA in the lab. Then the instructor
  - Escalating is not snitching, it's project management.

# Do you see why Team Contracts are important?

# Keep Improving

- Start
  - What should we begin doing?
- Stop
  - What is wasting our time?
  - What is causing friction?
  - What slowed us down?
- Continue
  - What is working well that we should not change?

# Why does all of this matter?

- For a safety-critical software, teamwork practices are not just *soft skills*, they are assurance evidence.
- Reviews are a team activity
  - Design reviews, code reviews, hazard analysis reviews. A regulator wants to see that someone other than the author checked it — and who, and when.
- Traceability is a team artifact
  - Requirement → design → code → test. It only exists if the requirements lead, the developer and the tester all wrote things down in the same place.
- Enforced diversity
  - Independent teams don't automatically produce independent errors. Diverse backgrounds and deliberate cross-checking do help
- Speaking up
  - Most of the famous failures we showed last week had someone who knew. Psychological safety is a safety property.

# Takeaways!

- Software is built by teams; the process models from earlier this week are executed by people who have to communicate
- Contribution imbalance is the strongest predictor of a poor project grade, for everyone on the team
- Storming is normal.
- Clear roles (with exactly one accountable person per task) prevent most “I thought you were doing it”
- Meet regularly, write decisions down, and make it safe to speak up
- Git branching, code review, a board with WIP limits and a definition of done are team practices, not tooling
- Spot free-riding and conflict early; escalate early; reflect after every deliverable
- The team contract is where all of this becomes real, due this week ☺

# Questions?

# SFWRENG / MECHTRON 3K04 Software Development

## Today: Project Overview

McMaster University | Engineering
Computing & Software

September 18th, 2026

Dr. Angela Zavaleta Bernuy

# Course Project

- Automated Dispensing Cabinet (ADC) + Hospital Monitoring System
- A safety-critical hospital medication management solution
- Teams of 4–5, all in the same lab section
- Three deliverables, each with a lab demo + a submission

# Why this project?

- Real-world safety-critical software: mistakes can harm patients
- Covers the full software development lifecycle, not just coding
- Emphasis on analysis, requirements, design, safety, and integration
- Two independently built systems must work together at the end

# Part A: Automated Dispensing Cabinet (ADC)

- The "software brain" of a secure cabinet at the point of care
- Operates locally in the ward; controls locks, drawers, sensors, actuators, local UI
- Must keep operating safely even when the network is down
- Users: nurses, pharmacists, technicians

# ADC: Scope and Capabilities

- In scope: access control, dispensing, inventory updates, event logging, fault detection, operational modes, external communication
- Out of scope: mechanical design, drawers/locks hardware, hospital databases, patient info systems, Monitoring System internals, medical decisions
- Assume: hardware exists, sensors/actuators have software interfaces, network available, credential capture available

# ADC: Key Scenarios and Constraints

- Scenario 1: Nurse authenticates → requests medication → ADC verifies, opens location, updates inventory, logs
- Scenario 2: Technician replenishes inventory
- Scenario 3: Drawer fails to close → ADC detects and responds
- Scenario 4: Communication lost → ADC continues locally, buffers data
- Constraints: authorized users only, accurate inventory, everything recorded, safe under comms failure, faults detected

# Part B: Hospital Monitoring System

- Centralized, human-facing application supervising one or more ADCs
- Does not touch hardware; provides the clinical and administrative view
- Still safety-critical: wrong user authorized, stale schedule, hidden alert, offline device treated as healthy
- Users: nurses, pharmacists, technicians, administrators

# Monitoring System: Scope and Capabilities

- In scope: ADC communication, inventory supervision, alarm management, historical data, reporting, operator dashboard, cabinet-health monitoring, request handling, configuration distribution
- Out of scope: ADC hardware, direct cabinet control, ADC software internals, patient info systems, network design
- Assume: multiple ADCs exist with communication interfaces, users have valid accounts

# Monitoring System: Key Scenarios and Constraints

- Scenario 1: Pharmacist reviews inventory across cabinets; system flags those needing attention
- Scenario 2: ADC sends an alarm → received, recorded, displayed
- Scenario 3: An ADC goes silent → detected, status updated
- Scenario 4: Pharmacist reviews historical dispensing activity
- Constraints: multi-ADC, consistent data, events stored, access controlled, comms problems detected, expandable

# Integration: The ADC <-> Monitoring Interface

- Interface requirements are intentionally NOT provided, you define them
- Information exchanged: operational status, inventory, alarms, maintenance, system health
- ADC team defines: what it provides, what it needs, comms requirements, local failure handling, synchronization
- Monitoring team defines: what it receives, what it sends, storage/sync requirements, handling offline ADCs
- Final goal: two independently designed systems communicate correctly

# Deliverable 1: Requirements and Scope

- Worth 10% of final grade
- Demo in lab, week of Oct 5 (70%) + requirements document (30%)
- Purpose: establish what the system must do and for whom, and model it clearly
- Document should be ~95% done at demo time

# Deliverable 1: Document Structure

- Introduction and system context
- Functional requirements: by capability, uniquely identified, atomic, verifiable
- Non-functional requirements: safety, security, reliability, availability, performance, auditability, usability
- User stories: "As a <role>, I want <goal> so that <benefit>" + Given/When/Then acceptance criteria (INVEST)
- High-level architecture diagram with the ADC ↔ Monitoring boundary marked
- UML: domain/class model, use case, activity (≥4 scenarios), state machine
- Traceability table: requirement → story → component → stakeholder
- Assumptions and decisions, including ethical and accessibility considerations

# Deliverable 1: What the TAs Look For in the Demo

- Requirements are clear, unambiguous, verifiable, traceable
- Well-formed user stories with testable acceptance criteria
- UML covers structure and behaviour, uses proper notation, matches the text
- Ethical thinking visibly shaped requirements (safety, privacy, access, auditability)
- Team can move between requirement → story → diagram
- Every member can speak to every part. TAs pick who answers
- Team can absorb an on-the-spot change (new actor, new story)

# Tips for Success

- If you can't describe how to verify a requirement, rewrite it
- Keep all views consistent, inconsistencies lose demo marks fastest
- Keep the architecture diagram conceptual; detailed decomposition comes in D2
- Use proper UML notation; neatness signals understanding
- Make assumptions explicit; ask TAs and ReQuest when unsure
- Set a stable ID scheme now, you'll trace design and tests to it later
- Rehearse so any member can defend any aspect

# Team Contracts and ReQuest

- Choose one of two team contract templates in your first lab
- Agree on communication, accountability, and group policies; it's a binding document for resolving conflict
- Upload signed contract to Avenue by tonight!
- ReQuest: an educational tool for practicing requirements elicitation and question-asking
- Must be used at least once; most useful before Deliverable 1
- Access details on Avenue SOON

# Getting Started This Week

- Form your team and tell your TAs
- Pick and sign a team contract
- Decide who's on ADC vs. Monitoring System
- Read both Part A and Part B, you'll need to understand the other side for integration
- Start listing assumptions and questions for ReQuest and the TAs
- Support: weekly labs, Piazza, and the best practices document

# Questions?